



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Основи проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Основи проектування.

Мета: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проєктується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області.

Тема проєкту: Mind-mapping software

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	2
Хід роботи	4
Діаграма варіантів використання	4
Сценарії використання	5
Діаграма класів	8
Зв'язки між класами	9
Структура БД	9
Висновок	9
Вихідні коди класів	9

Теоретичні відомості

Мова UML – загальноцільова мова візуального моделювання, яка розроблена для специфікації, візуалізації, проєктування та документування

компонентів програмного забезпечення, бізнес-процесів та інших систем. Мова UML є досить строгим та потужним засобом моделювання, який може бути ефективно використаний для побудови концептуальних, логічних та графічних моделей складних систем різного цільового призначення. Ця мова увібрала в себе найкращі якості та досвід методів програмної інженерії, які з успіхом використовувалися протягом останніх років при моделюванні великих та складних систем.

Діаграма (diagram) – графічне уявлення сукупності елементів моделі у формі зв'язкового графа, вершинам і ребрам (дугам) якого приписується певна семантика. Нотація канонічних діаграм є основним засобом розробки моделей мовою UML.

Діаграма варіантів використання (Use-Cases Diagram) – це UML діаграма за допомогою якої у графічному вигляді можна зобразити вимоги до системи, що розробляється. Діаграма варіантів використання – це вихідна концептуальна модель проєктованої системи, вона не описує внутрішню побудову системи.

Сценарії використання – це текстові уявлення тих процесів, які відбуваються при взаємодії користувачів системи та самої системи. Вони є чітко формалізованими, покроковими інструкціями, що описують той чи інший процес у термінах кроків досягнення мети. Сценарії використання однозначно визначають кінцевий результат.

Діаграми класів використовуються при моделюванні програмних систем найчастіше. Вони є однією із форм статичного опису системи з погляду її проєктування, показуючи її структуру. Діаграма класів не відображає динамічної поведінки об'єктів зображених на ній класів. На діаграмах класів показуються класи, інтерфейси та відносини між ними.

Клас – це основний будівельний блок програмної системи. Це поняття є і в мовах програмування, тобто між класами UML та програмними класами є відповідність, що є основою для автоматичної генерації програмних кодів або для виконання реінжинірингу. Кожен клас має назву, атрибути та операції. Клас на діаграмі показується як прямокутник, розділений на 3 області. У верхній міститься назва класу, у середній – опис атрибутів (властивостей), у нижній – назви операцій – послуг, що надаються об'єктами цього класу.

Логічна модель бази даних є структурою таблиць, уявлень, індексів та інших логічних елементів бази даних, що дозволяють власне програмування та використання бази даних.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Проаналізувати тему та спроектувати діаграму варіантів використання відповідно до обраної теми лабораторного циклу.
- Спроектувати діаграму класів предметної області.
- Вибрати 3 варіанти використання та написати за ними сценарії використання.
- На основі спроектованої діаграми класів предметної області розробити основні класи та структуру бази даних системи. Класи даних повинні реалізувати шаблон Repository для взаємодії з базою даних.
- Нарисувати діаграму класів для реалізованої частини системи.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму варіантів використання відповідно, діаграму класів системи, вихідні коди класів системи, а також зображення структури бази даних.

Діаграма варіантів використання

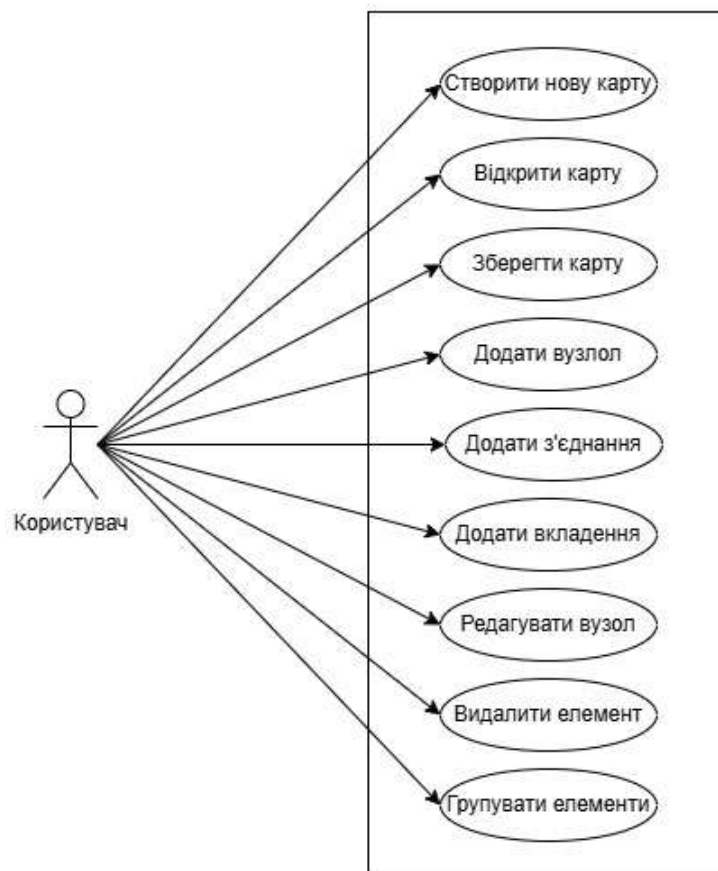


Рис.1. Діаграма варіантів використання

Сценарії використання (Use case Scenarios)

1. Сценарій: Створити нову карту

Передумови: Користувач знаходиться на головному екрані додатку

Постумови: Нова карта створена та збережена в системі. Вона з'явиться у загальному списку карт

Взаємодіючі сторони: Користувач. Система

Короткий опис: Цей варіант використання описує процес додавання Користувачем нової карти до списку

Основний потік подій:

Користувач ініціює створення карти (наприклад натискає кнопку «Створити нову карту»)

Система відображає форму/поля для введення параметрів нової карти

Користувач вводить назву карти

Користувач підтверджує створення (натискає клавішу “Enter” або кнопку «Створити»)

Система валідує введені дані (перевіряє, чи назва не порожня)

Система зберігає нову карту в базі даних

Система оновлює список карт Користувача, відображаючи новостворену карту в списку

Система автоматично відкриває новостворену карту в робочій області

Варіант використання завершується

Винятки:

Виняток 1 (Крок 5): Назва карти порожня.

Система не зберігає карту

Система виводить повідомлення про помилку: «Назва завдання не може бути порожньою»

Користувач повертається до Кроку 3

2. Сценарій: Додати з'єднання

Передумови: Користувач бачить в робочому просторі відкриту карту яка містить принаймні два вузли

Постумови: Нове з'єднання створено та збережено в системі. Воно буде відображено в робочому просторі між двома обраними вузлами

Взаємодіючі сторони: Користувач. Система

Короткий опис: Цей варіант використання описує процес створення зв'язку між двома обраними користувачем вузлами

Основний потік подій:

Користувач здійснює виділення вузлів у робочій області відкритої карти

Користувач ініціює створення зв'язку (наприклад, натискає кнопку «Створити зв'язок»)

Система перевіряє кількість виділених вузлів.

Система зберігає дані про зв'язок в базі даних

Система оновлює візуальний інтерфейс робочої області відображаючи новий зв'язок між вузлами

Система знімає виділення з вузлів між якими було створено зв'язок

Варіант використання завершується

Винятки:

Виняток 1 (Крок 3): Виділено не два вузли.

Система не зберігає дані про зв'язок

Система виводить повідомлення про помилку: «Оберіть рівно два вузли для створення зв'язку»

Якщо кількість виділених вузлів відмінна від нуля або двох, виділення знімається

Користувач повертається до Кроку 1

3. Сценарій: Видалити елемент

Передумови: Користувач бачить в робочому просторі відкриту карту яка містить принаймні один вузол або більшу кількість вузлів і принаймні один зв'язок між деякими з вузлів

Постумови: Виділений користувачем елемент/елементи (вузол/зв'язок) видалені з системи, та не будуть відображені у візуальному інтерфейсі робочої області

Взаємодіючі сторони: Користувач. Система

Короткий опис: Цей варіант використання описує процес видалення елементу (вузол/зв'язок)

Основний потік подій:

Користувач виділяє відмінну від нуля кількість елементів (вузлів/зв'язків) у робочій області відкритої карти

Елемент візуального інтерфейсу (наприклад кнопка «Видалити елемент(и)») що буде використано для ініціації видалення стає доступним для Користувача (Отримує можливість ініціювати видалення при взаємодії і змінює візуальні характеристики або просто з'являється у панелі інструментів)

Користувач ініціює видалення елементів (вузлів/зв'язків) (наприклад, натискає кнопку «Видалити елемент(и)»)

Система видаляє дані про обрані елементи з бази даних, якщо серед обраних елементів були вузли, то їх зв'язки видаляються також

Система оновлює візуальний інтерфейс робочої області видаляючи виділені елементи

Варіант використання завершується

Діаграма класів (Class Diagram)

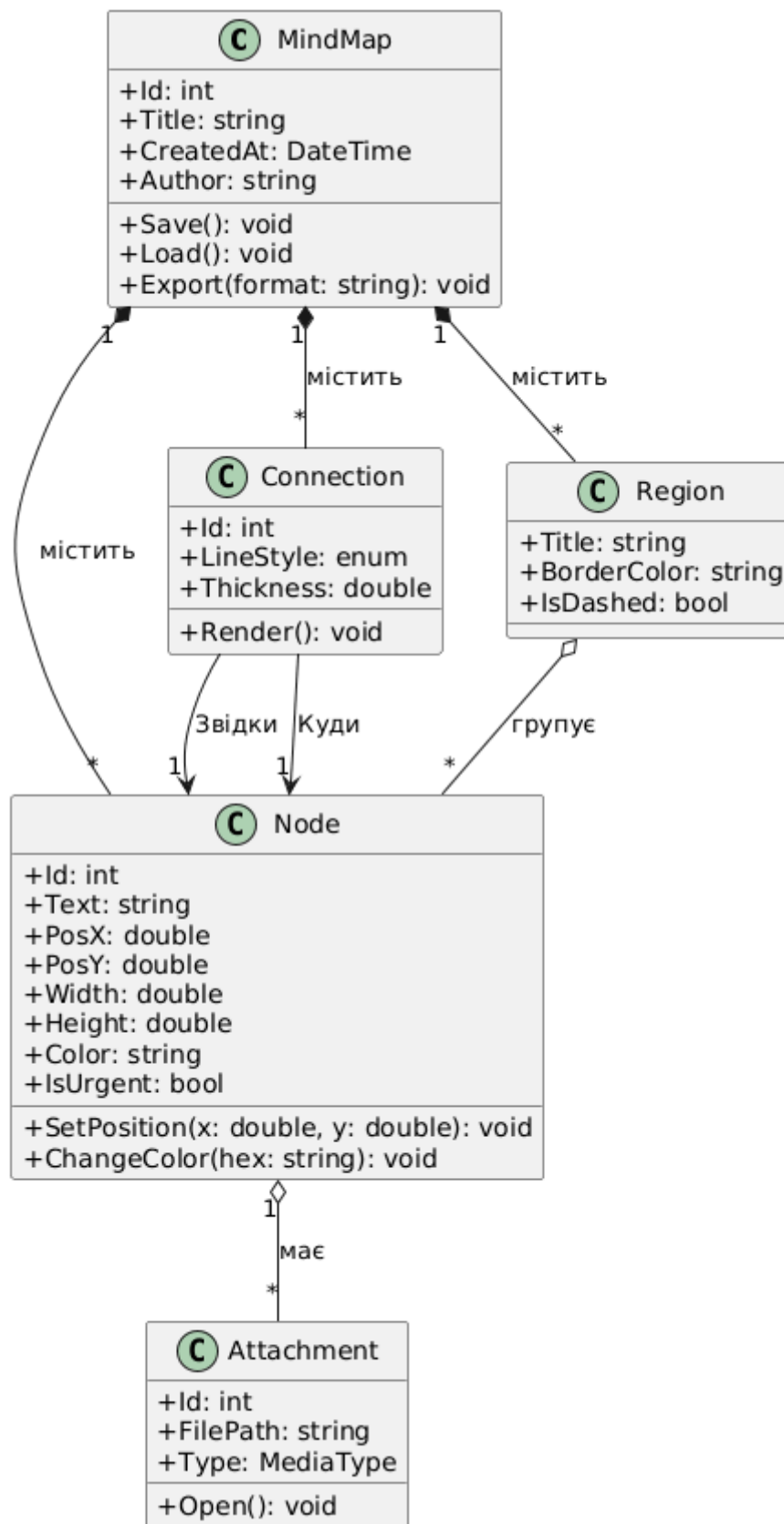


Рис. 2. Діаграма класів

Зв'язки між класами

MindMap – Node

Композиція. MindMap володіє об'єктами Node

MindMap – Connection

Композиція. MindMap володіє об'єктами Connection

MindMap – Region

Композиція. MindMap володіє об'єктами Region

Region – Node

Агрегація. Region посилається на Node, але Node може існувати як самостійна сутність

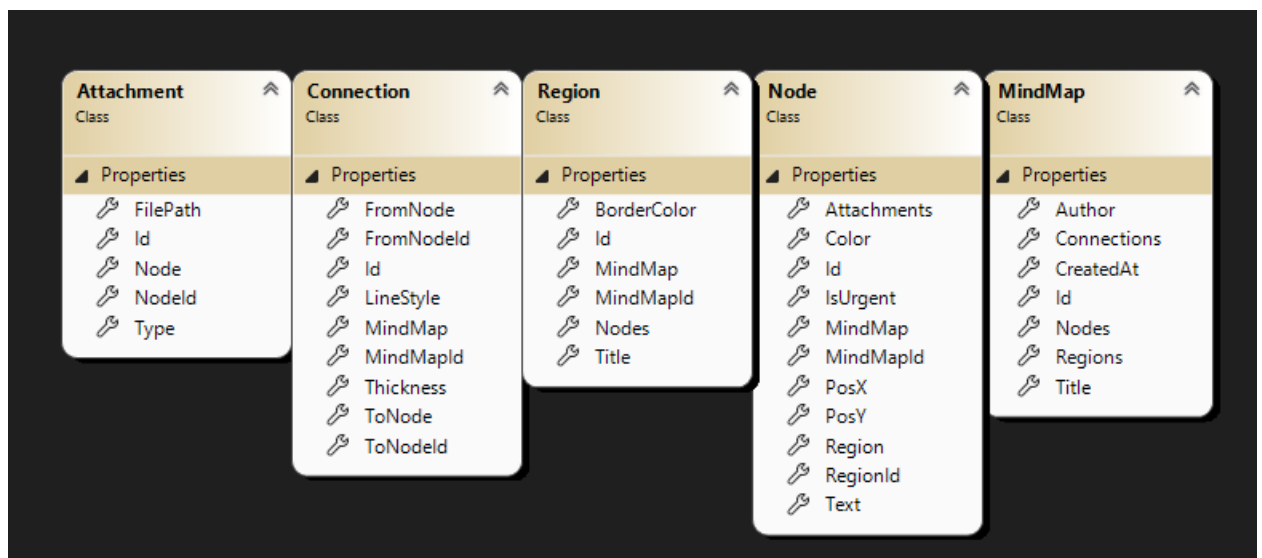
Node – Attachment

Агрегація. Node може посилатися на Attachment, але Attachment може існувати сам по собі.

Connection – Node

Асоціація. Connection посилається на два об'єкти Node

Реалізація структури через БД



Висновок: Протягом виконання лабораторної роботи було визначено основні сутності предметної області, створена логічна модель даних, представлено реалізацію основних функціональних вимог через сценарії використання

Вихідні коди класів

MindMap

```
using System;
```

```
using System.Collections.Generic;
```

```
namespace MindMapApp.Entities
```

```
{
```

```
    public class MindMap
```

```
    {
```

```
        public int Id { get; set; }
```

```
        public string Title { get; set; } = string.Empty;
```

```
        public DateTime CreatedAt { get; set; } = DateTime.Now;
```

```
        public string Author { get; set; } = string.Empty;
```

```
        public virtual ICollection<Node> Nodes { get; set; } = new List<Node>();
```

```
        public virtual ICollection<Connection> Connections { get; set; } = new  
List<Connection>();
```

```
        public virtual ICollection<Region> Regions { get; set; } = new  
List<Region>();
```

```
    }
```

```
}
```

Node

```
using System.Collections.Generic;
```

```
using System.Net.Mail;
```

```
namespace MindMapApp.Entities
```

```

{
    public class Node
    {
        public int Id { get; set; }

        public string Text { get; set; } = string.Empty;


        public double PosX { get; set; }

        public double PosY { get; set; }


        public string Color { get; set; } = "#000000";

        public bool IsUrgent { get; set; }


        public int MindMapId { get; set; }

        public virtual MindMap MindMap { get; set; }


        public int? RegionId { get; set; }

        public virtual Region Region { get; set; }


        public virtual ICollection<Attachment> Attachments { get; set; } = new
        List<Attachment>();
    }
}

```

Region

```

using System.Collections.Generic;

```

```
namespace MindMapApp.Entities
```

```
{
```

```
    public class Region
```

```
    {
```

```
        public int Id { get; set; }
```

```
        public string Title { get; set; } = string.Empty;
```

```
        public string BorderColor { get; set; } = "#FF0000";
```

```
        public int MindMapId { get; set; }
```

```
        public virtual MindMap MindMap { get; set; }
```

```
        public virtual ICollection<Node> Nodes { get; set; } = new List<Node>();
```

```
    }
```

```
}
```

Connection

```
namespace MindMapApp.Entities
```

```
{
```

```
    public class Connection
```

```
    {
```

```
        public int Id { get; set; }
```

```
        public string LineStyle { get; set; } = "Solid"; // "Solid", "Dashed" і т.д.
```

```
        public double Thickness { get; set; } = 1.0;
```

```
        public int MindMapId { get; set; }
```

```
public virtual MindMap MindMap { get; set; }
```

```
public int FromNodeId { get; set; }
```

```
public virtual Node FromNode { get; set; }
```

```
public int ToNodeId { get; set; }
```

```
public virtual Node ToNode { get; set; }
```

```
}
```

```
}
```

Attachment

```
namespace MindMapApp.Entities
```

```
{
```

```
public enum AttachmentType
```

```
{
```

```
    Image,
```

```
    Video,
```

```
    Document
```

```
}
```

```
public class Attachment
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string FilePath { get; set; } = string.Empty;
```

```
    public AttachmentType Type { get; set; }
```

```
    public int NodeId { get; set; }
```

```
public virtual Node Node { get; set; }  
}  
}
```